
NAME: S.DHARSHINI

REG NO: 192424258

SUB CODE: DSA0502

SUB NAME: QUERY PROCESSING FOR DATA SCIENCE

CO2 ASSESSMENT TOOL 1

DATE: 1.08.26

Scenario 5: Hotel Reservation and Booking System

A hotel chain operates multiple branches across different cities. Customers can book rooms online or through customer service representatives. The hotel management wants a centralized system to manage customer information, room availability, reservations, payments, and employee details. Different room categories such as standard, deluxe, and suite must be maintained. The system should track check-in and check-out details, generate invoices, and maintain customer booking history. Analyze the requirements and design a database structure that minimizes redundancy and supports efficient querying. Use Python to connect to the database and demonstrate room booking, reservation updates, and customer management functionalities.

AIM: To design and implement a Hotel Reservation and Booking System using MySQL and Python to manage hotel branches, customers, room reservations, payments, employees, and booking history efficiently.

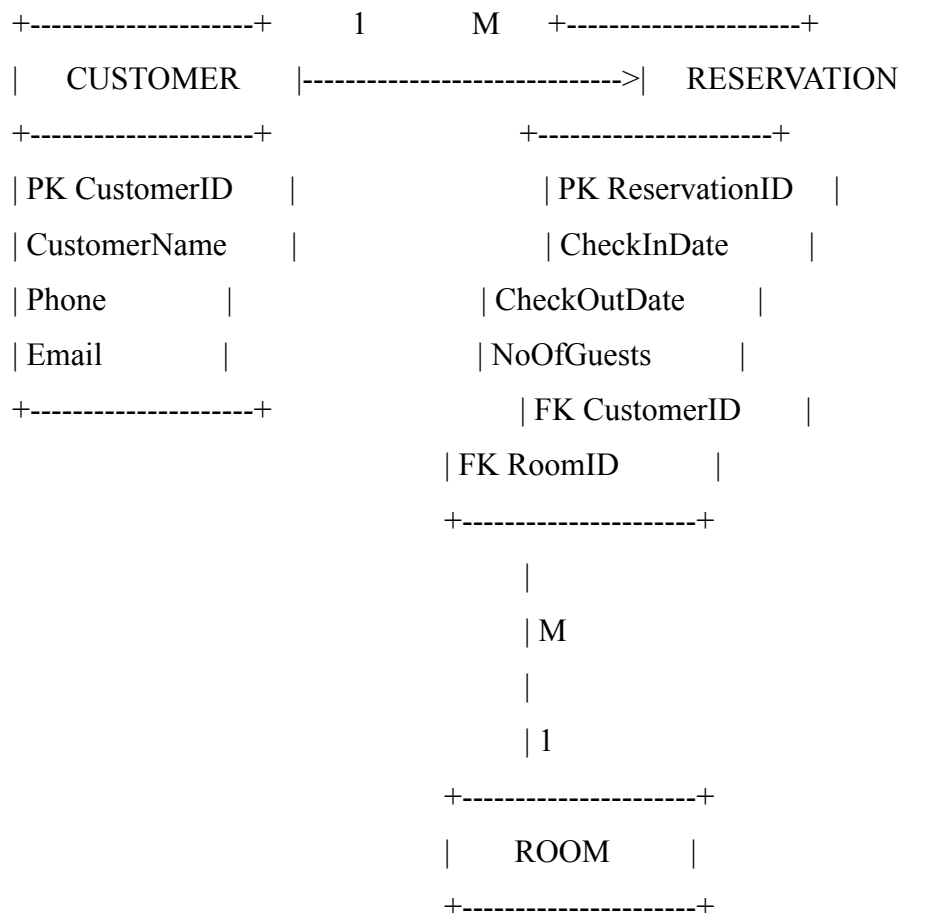
PROBLEM STATEMENT: Develop a centralized hotel reservation system that stores customer details, room availability, reservations, payments, employee information, and booking history while minimizing redundancy through database normalization.

PROCEDURE:

- Connect Python to the MySQL database.
- Create the required database and tables.
- Insert sample records into each table.
- Perform customer registration and room booking.
- Update or cancel reservations as required.
- Display booking details, payment information, and customer booking history.

ER DIAGRAM:

HOTEL RESERVATION SYSTEM (EER DIAGRAM)



PK RoomID	
RoomNumber	
RoomType	
PricePerNight	
Status	

+-----+

|

| M

|

| 1

+-----+

HOTEL	
-------	--

+-----+

PK HotelID	
------------	--

HotelName	
-----------	--

Address	
---------	--

Phone	
-------	--

+-----+

|

| 1

|

| M

+-----+

EMPLOYEE	
----------	--

+-----+

PK EmployeeID	
---------------	--

EmployeeName	
--------------	--

Designation	
-------------	--

Phone	
-------	--

FK HotelID	
------------	--

+-----+

OUTPUT:

```
mysql> CREATE DATABASE hotelDB;
Query OK, 1 row affected (1.234 sec)

mysql> USE hotelDB;
Database changed

mysql> CREATE TABLE Branch(
  -> BranchID INT PRIMARY KEY AUTO_INCREMENT,
  -> BranchName VARCHAR(100),
  -> City VARCHAR(100),
  -> Address VARCHAR(200),
  -> Phone VARCHAR(15)
) ENGINE=InnoDB;
Query OK, 0 rows affected (1.287 sec)

mysql> desc Branch;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| BranchID | int | NO | PRI | NULL | auto_increment |
| BranchName | varchar(100) | YES | NULL | NULL |
| City | varchar(100) | YES | NULL | NULL |
| Address | varchar(200) | YES | NULL | NULL |
| Phone | varchar(15) | YES | NULL | NULL |
+-----+-----+-----+-----+-----+
1 rows in set (0.421 sec)

mysql> CREATE TABLE Customer(
  -> CustomerID INT PRIMARY KEY AUTO_INCREMENT,
  -> CustomerName VARCHAR(100),
  -> Phone VARCHAR(15),
  -> Email VARCHAR(100),
  -> Address VARCHAR(200)
) ENGINE=InnoDB;
Query OK, 0 rows affected (0.407 sec)

mysql> desc Customer;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| CustomerID | int | NO | PRI | NULL | auto_increment |
| CustomerName | varchar(100) | YES | NULL | NULL |
| Phone | varchar(15) | YES | NULL | NULL |
+-----+-----+-----+-----+-----+
1 rows in set (0.412 sec)

mysql> CREATE TABLE Room(
  -> RoomID INT PRIMARY KEY AUTO_INCREMENT,
  -> BranchID INT,
  -> RoomType VARCHAR(20),
  -> Price DECIMAL(10,2),
  -> Availability VARCHAR(20),
  -> FOREIGN KEY(BranchID) REFERENCES Branch(BranchID)
) ENGINE=InnoDB;
Query OK, 0 rows affected (0.389 sec)

mysql> desc Room;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| RoomID | int | NO | PRI | NULL | auto_increment |
| BranchID | int | YES | M | NULL |
| RoomType | varchar(20) | YES | NULL | NULL |
| Price | decimal(10,2) | YES | NULL | NULL |
| Availability | varchar(20) | YES | NULL | NULL |
+-----+-----+-----+-----+-----+
1 rows in set (0.418 sec)

mysql> CREATE TABLE Reservation(
  -> ReservationID INT PRIMARY KEY AUTO_INCREMENT,
  -> CustomerID INT,
  -> RoomID INT,
  -> CheckIn DATE,
  -> CheckOut DATE,
  -> BookingDate DATE,
  -> Status VARCHAR(10),
  -> FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
  -> FOREIGN KEY(RoomID) REFERENCES Room(RoomID)
) ENGINE=InnoDB;
Query OK, 0 rows affected (0.407 sec)
```

```
mysql> desc Reservation;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| ReservationID | int | NO | PRI | NULL | auto_increment |
| CustomerID | int | YES | M | NULL |
| RoomID | int | YES | M | NULL |
| CheckIn | date | YES | NULL | NULL |
| CheckOut | date | YES | NULL | NULL |
| BookingDate | date | YES | NULL | NULL |
| Status | varchar(10) | YES | NULL | NULL |
+-----+-----+-----+-----+-----+
1 rows in set (0.407 sec)

mysql> CREATE TABLE Payment(
  -> PaymentID INT PRIMARY KEY AUTO_INCREMENT,
  -> ReservationID INT,
  -> Amount DECIMAL(10,2),
  -> PaymentMethod VARCHAR(20),
  -> PaymentDate DATE,
  -> FOREIGN KEY(ReservationID) REFERENCES Reservation(ReservationID)
) ENGINE=InnoDB;
Query OK, 0 rows affected (0.404 sec)

mysql> desc Payment;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| PaymentID | int | NO | PRI | NULL | auto_increment |
| ReservationID | int | YES | M | NULL |
| Amount | decimal(10,2) | YES | NULL | NULL |
| PaymentMethod | varchar(20) | YES | NULL | NULL |
| PaymentDate | date | YES | NULL | NULL |
+-----+-----+-----+-----+-----+
1 rows in set (0.417 sec)

mysql> CREATE TABLE Employee(
  -> EmployeeID INT PRIMARY KEY AUTO_INCREMENT,
  -> EmployeeName VARCHAR(100),
  -> Designation VARCHAR(100),
  -> Salary DECIMAL(10,2),
  -> BranchID INT,
  -> FOREIGN KEY(BranchID) REFERENCES Branch(BranchID)
) ENGINE=InnoDB;
Query OK, 0 rows affected (0.413 sec)
```

```
mysql> SELECT * FROM Branch;
+-----+-----+-----+-----+
| BranchID | BranchName | City | Address | Phone |
+-----+-----+-----+-----+
| 1 | Channel Grand | Chennai | Anna Nagar | 9876543210 |
| 2 | Collaborative Inn | Coimbatore | RS Puram | 9876543210 |
+-----+-----+-----+-----+
2 rows in set (0.085 sec)

mysql> SELECT * FROM Customer;
+-----+-----+-----+-----+
| CustomerID | CustomerName | Phone | Email | Address |
+-----+-----+-----+-----+
| 1 | Rahul | 9876543212 | rahul@gmail.com | Chennai |
| 2 | Priya | 9876543211 | priya@gmail.com | Madurai |
+-----+-----+-----+-----+
2 rows in set (0.085 sec)

mysql> SELECT * FROM Room;
+-----+-----+-----+-----+
| RoomID | BranchID | RoomType | Price | Availability |
+-----+-----+-----+-----+
| 1 | 1 | Standard | 2000.00 | Available |
| 2 | 1 | Deluxe | 3000.00 | Available |
| 3 | 2 | Suite | 4000.00 | Available |
+-----+-----+-----+-----+
3 rows in set (0.064 sec)

mysql> USE hotelDB;
Database changed

mysql> SELECT * FROM Customer;
+-----+-----+-----+-----+
| CustomerID | CustomerName | Phone | Email | Address |
+-----+-----+-----+-----+
| 1 | Rahul | 9876543212 | rahul@gmail.com | Chennai |
| 2 | Priya | 9876543211 | priya@gmail.com | Madurai |
| 3 | Arun | 9876543213 | arun@gmail.com | Chennai |
| 4 | Priya | 9876543212 | priya@gmail.com | Madurai |
| 5 | Arun | 9876543213 | arun@gmail.com | Chennai |
+-----+-----+-----+-----+
5 rows in set (0.086 sec)

mysql> SELECT * FROM Reservation;
+-----+-----+-----+-----+
| ReservationID | CustomerID | RoomID | CheckIn | CheckOut | BookingDate | Status |
+-----+-----+-----+-----+
| 1 | 1 | 1 | 2020-08-05 | 2020-08-08 | 2020-07-31 | Booked |
+-----+-----+-----+-----+
1 rows in set (0.067 sec)

mysql> SELECT * FROM Room;
+-----+-----+-----+-----+
| RoomID | BranchID | RoomType | Price | Availability |
+-----+-----+-----+-----+
| 1 | 1 | Standard | 2000.00 | Booked |
| 2 | 1 | Deluxe | 3000.00 | Available |
| 3 | 2 | Suite | 4000.00 | Available |
| 4 | 1 | Deluxe | 3000.00 | Available |
| 5 | 2 | Suite | 4000.00 | Available |
| 6 | 2 | Suite | 4000.00 | Available |
+-----+-----+-----+-----+
6 rows in set (0.085 sec)

mysql> SELECT * FROM Reservation;
+-----+-----+-----+-----+
| ReservationID | CustomerID | RoomID | CheckIn | CheckOut | BookingDate | Status |
+-----+-----+-----+-----+
| 1 | 1 | 1 | 2020-08-05 | 2020-08-08 | 2020-07-31 | Checked In |
+-----+-----+-----+-----+
1 rows in set (0.085 sec)
```

CONCLUSION: The system provides a centralized solution for hotel management by reducing data redundancy, improving reservation handling, maintaining room availability, and enabling efficient querying through a well-designed relational database.

Scenario 10: Smart Transportation and Ticket Booking System

A transportation company operates buses across multiple cities. Passengers can search routes, book tickets, cancel reservations, and make online payments. The company must maintain information about buses, routes, schedules, drivers, passengers, and ticket bookings. Each bus operates on multiple routes, and passengers may make several bookings over time. Management wants to analyze passenger demand, route utilization, and revenue generation. Design a database system that efficiently stores transportation data and supports ticket booking operations. Use Python SQL libraries to connect to the database and perform CRUD operations for route management, passenger registration, and ticket reservations.

AIM: To design and implement a Smart Transportation and Ticket Booking System using MySQL and Python for managing buses, routes, drivers, passengers, ticket bookings, schedules, and payments while supporting efficient CRUD operations.

PROBLEM STATEMENT: A transportation company operates buses across multiple cities. Passengers need to search routes, book tickets, cancel reservations, and make online payments. The company also needs to maintain details of buses, routes, schedules, drivers, passengers, and bookings. The system should support efficient storage of transportation data, reduce redundancy, and provide quick access to booking and passenger information through a centralized database.

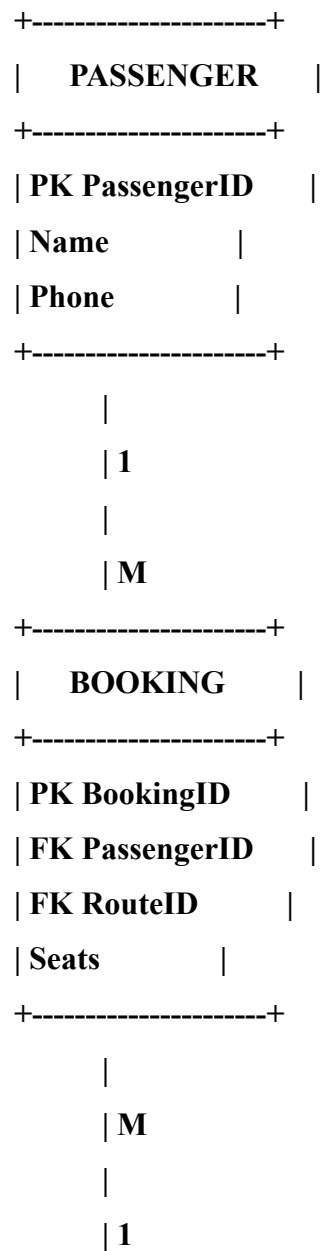
PROCEDURE:

- Open MySQL and create the TransportDB database.
- Create all required tables with primary and foreign keys.
- Insert sample records into the Bus, Route, Driver, Passenger, and Schedule tables.
- Open the project folder in VS Code.
- Connect Python to the MySQL database using `mysql.connector`.
- Execute the Passenger Registration program.
- Execute the Ticket Booking program.
- Update booking details using the Update Booking module.

- Display booking details using SQL JOIN queries.
- Verify the output and database records.

ER DIAGRAM:

SMART TRANSPORTATION AND TICKET BOOKING SYSTEM



ROUTE

PK RouteID

Source

Destination

Fare

FK BusID

--

M

--

1

BUS

PK BusID

BusNumber

BusType

Capacity

FK DriverID

--

M

--

1

DRIVER

PK DriverID

DriverName

| Phone |
| LicenseNumber |
+-----+

OUTPUT:

The first screenshot shows the MySQL command line client with the following commands and output:

```
mysql> CREATE DATABASE TransportDB;
Query OK, 1 row affected (0.198 sec)

mysql> USE TransportDB;
Database changed

mysql> CREATE TABLE Bus
  ( BusID INT PRIMARY KEY AUTO_INCREMENT,
  RouteName VARCHAR(100),
  RouteType VARCHAR(50),
  Capacity INT
  );
Query OK, 8 rows affected (0.307 sec)

mysql> CREATE TABLE Route
  ( RouteID INT PRIMARY KEY AUTO_INCREMENT,
  Source VARCHAR(50),
  Destination VARCHAR(50),
  Distance INT
  );
Query OK, 8 rows affected (0.492 sec)

mysql> CREATE TABLE Driver
  ( DriverID INT PRIMARY KEY AUTO_INCREMENT,
  LicenseNumber VARCHAR(50),
  Phone VARCHAR(50),
  Email VARCHAR(50)
  );
Query OK, 8 rows affected (0.273 sec)

mysql> CREATE TABLE Passenger
  ( PassengerID INT PRIMARY KEY AUTO_INCREMENT,
  PassengerName VARCHAR(50),
  Phone VARCHAR(50),
  Email VARCHAR(50)
  );
Query OK, 8 rows affected (0.273 sec)
```

The second screenshot shows the MySQL command line client with the following commands and output:

```
mysql> CREATE TABLE Booking
  ( BookingID INT PRIMARY KEY AUTO_INCREMENT,
  PassengerID INT,
  ScheduleID INT,
  SeatNo INT,
  BookingDate DATE,
  Status VARCHAR(50),
  FOREIGN KEY(PassengerID) REFERENCES Passenger(PassengerID),
  FOREIGN KEY(ScheduleID) REFERENCES Schedule(ScheduleID)
  );
Query OK, 8 rows affected (0.488 sec)

mysql> CREATE TABLE Payment
  ( PaymentID INT PRIMARY KEY AUTO_INCREMENT,
  PassengerID INT,
  BookingID INT,
  Amount DECIMAL(10,2),
  PaymentDate DATE,
  FOREIGN KEY(PassengerID) REFERENCES Passenger(PassengerID),
  FOREIGN KEY(BookingID) REFERENCES Booking(BookingID)
  );
Query OK, 8 rows affected (0.810 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_TransportDB |
+-----+
| booking                |
| bus                    |
| driver                 |
| passenger              |
| payment                |
| route                  |
| schedule                |
+-----+
```

The screenshot shows the MySQL command line client with the following commands and output:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_TransportDB |
+-----+
| booking                |
| bus                    |
| driver                 |
| passenger              |
| payment                |
| route                  |
| schedule                |
+-----+
7 rows in set (0.015 sec)

mysql> USE TransportDB;
Database changed

mysql> SELECT * FROM Passenger;
+-----+
| PassengerID | PassengerName | Phone          | Email          |
+-----+
| 1           | Rahul         | 9876543212    | rahul@gmail.com |
| 2           | Sai           | 9876543210    | sai@gmail.com   |
+-----+
2 rows in set (0.006 sec)

mysql> SELECT * FROM Booking;
+-----+
| BookingID | PassengerID | ScheduleID | SeatNo | BookingDate | Status |
+-----+
| 1         | 1           | 2          | 12     | 2020-07-31  | Booked |
+-----+
1 row in set (0.006 sec)

mysql> SELECT * FROM Booking;
+-----+
| BookingID | PassengerID | ScheduleID | SeatNo | BookingDate | Status |
+-----+
| 1         | 1           | 2          | 12     | 2020-07-31  | Cancelled |
+-----+
1 row in set (0.005 sec)

mysql>
```

The screenshot shows a Python IDE with the following code in the `booking.py` file:

```
1 import mysql.connector
2 from datetime import date
3
4 # Connect to MySQL
5 con = mysql.connector.connect(
6     host="localhost",
7     user="root",
8     password="1234",
9     database="TransportDB"
10 )
11
12 cursor = con.cursor()
13
14 # Get booking details
```

The terminal output shows the following message:

```
Booking Updated Successfully on System
```

The output also shows the following details:

```
----- TRANSPORT BOOKING DETAILS -----
Booking ID      : 1
Passenger Name  : sai
Bus Name       : Express
Source         : Chennai
Destination    : Madurai
Travel Date    : 2020-08-10
Seat Number    : 12
Status        : Cancelled
```

CONCLUSION: The Smart Transportation and Ticket Booking System provides a centralized and efficient solution for managing transportation services. It enables easy passenger registration, route management, ticket booking, and booking updates while ensuring data consistency through

a relational database. Python and MySQL together provide a simple and effective platform for implementing the system.